Solution Architecture

Date	7 July2026
Team ID	xxx
Project Name	SmartLender
Maximum Marks	5 Marks

Solution Architecture Diagram

The diagram below maps Smart Lender's components onto the Presentation, API Gateway, Application, and Data layers.

Presentation / Client Layer
Web App (Flask + Jinja2 templates) — Applicant Form, Result Page, Batch Upload UI

↓↑

API Gateway / Load Balancer
Flask Routing Layer (app.py) — served via IBM Cloud Code Engine / Nginx

↓↑

Auth Service	**Core Logic Service**	**External API Integrations**
(Optional) Officer login / session control for internal use	*Preprocessing Pipeline + XGBoost Prediction Engine*	*Batch CSV/Excel Upload Processor, Notification/Email Service*

↓↑

Data / Storage Layer
Model Artifacts (xgboost_model.pkl, encoder.pkl, scaler.pkl), Uploads Directory, Applicant Datasets (Kaggle/UCI CSV)

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Web-based interface where credit officers and applicants submit applicant details, view single prediction results (approval, likelihood, risk label), and upload batch CSV/Excel files for multiple applicants.	HTML, CSS, JavaScript, Flask (Jinja2 templates) — index.html, result.html, batch.html
API Gateway	Single entry point that routes incoming HTTP requests to the correct Flask route (single prediction, batch upload, results), handles request/response formatting, and can sit behind a reverse proxy on IBM Cloud.	Flask routing (app.py), Werkzeug, Nginx / IBM Cloud Code Engine ingress
Microservices (Core Logic Service)	Cleans and encodes raw applicant input, scales numerical features, and runs the trained XGBoost model to generate an approval prediction, likelihood score, and risk category (Low/Medium/High). Also processes batch CSV uploads record-by-record.	Python, scikit-learn, pandas, numpy, XGBoost, pickle/joblib (xgboost_model.pkl, encoder.pkl, scaler.pkl)
Database	Stores serialized model artifacts, uploaded batch files, and (optionally) applicant records and prediction logs for auditability and retraining.	Local filesystem (model/, uploads/) for MVP; PostgreSQL or IBM Cloud Object Storage for scaled production use